Ollscoil Teicneolaíochta an Oirdheiscirt
South East Technological University

AI PRINCIPLES AND PRACTICE · WEEK 7 OF 12

How a Machine Learns, and How It Is Wired

Guess, check, blame, nudge

Lecturer: Indrakshi

Today, in one sentence

A machine learns by guessing, checking, and nudging its numbers, thousands of times.

GUESS — make a prediction · forward

› **CHECK** — how wrong was it? · loss

› **BLAME** — which dials caused it? · backward

› **NUDGE** — one small step downhill · update

↻ **then round again, thousands of times**

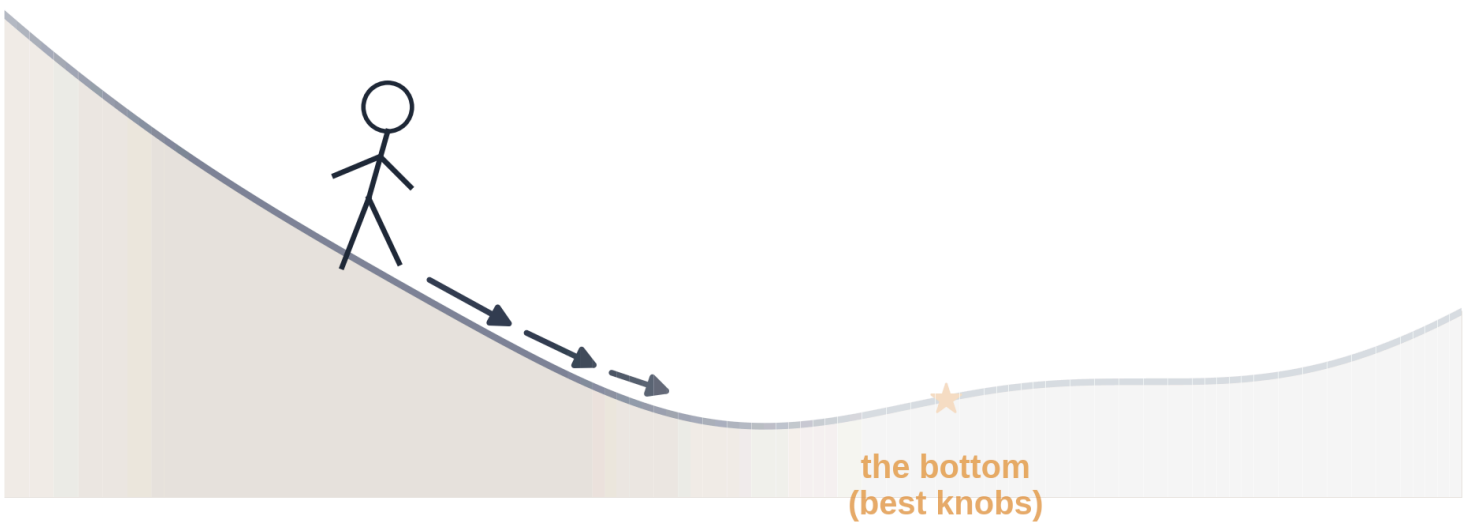
Today’s route

1. What “learning” means
2. **Guess** with one neuron
3. **Guess** with a network: how it is wired
4. **Check**: how wrong was it?
5. **Nudge**: walking downhill
6. **Blame**: passing it backwards
7. The loop, running

A hillwalker in fog

You can’t see the bottom.

But you can feel which way the ground slopes.



Feel the slope under your feet. Take a step downhill. Repeat.

PART 1

What “learning” means

A model is a formula with dials

GUESS
make a prediction · forward

›

CHECK
how wrong was it? · loss

›

BLAME
which dials caused it? · backward

›

NUDGE
one small step downhill · update

↻ **then round again, thousands of times**

A model is a formula with dials

THE FORMULA

$$\hat{y} = ax + b$$

IN WORDS

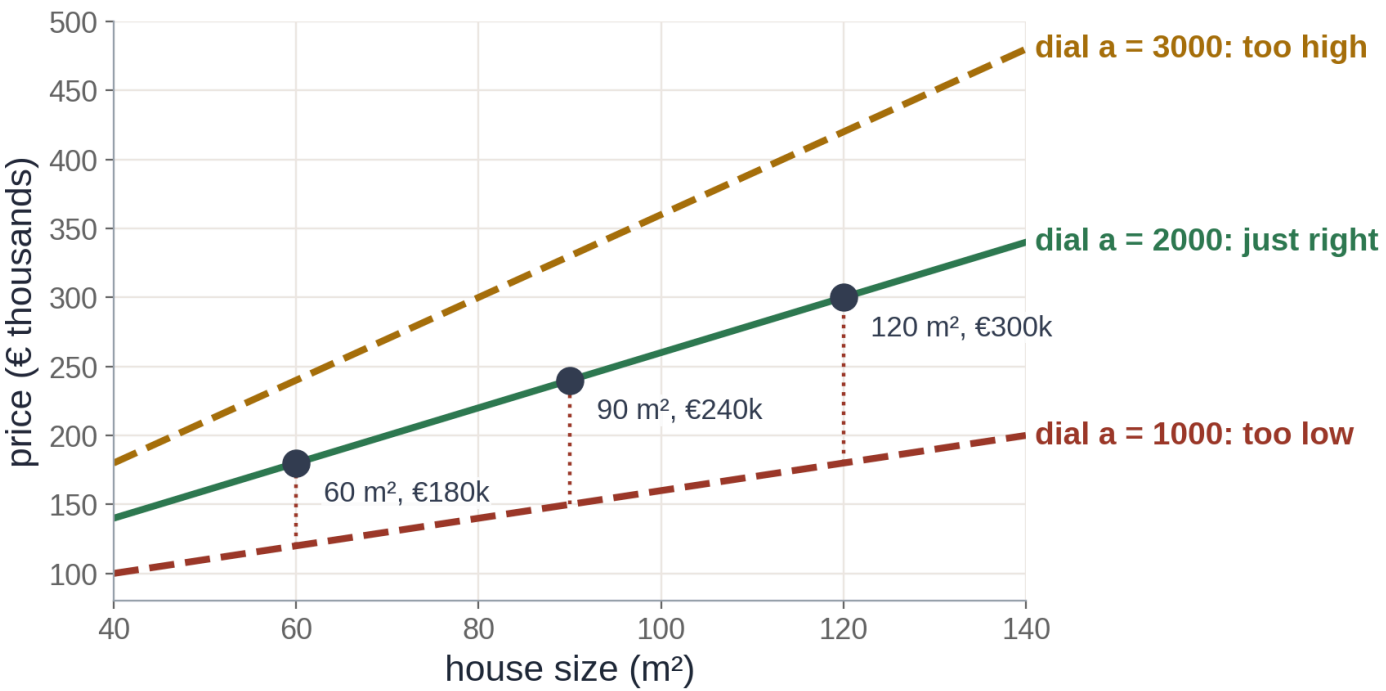
price = a × size + base price

WITH NUMBERS

€240,000 = 2,000 × 90 m² + €60,000

$\hat{y}$ (“y-hat”) is the guess. **a** and **b** are the dials the computer learns.

Three houses, three settings of dial a



Two words you will hear every week

Parameters

The dials **the computer** turns: a and b.

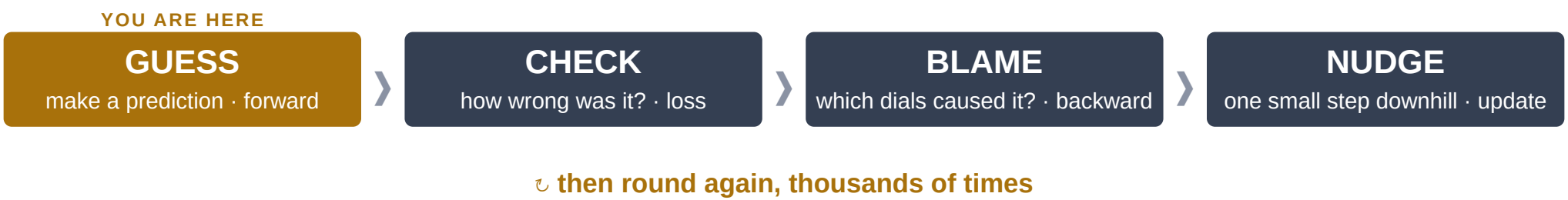
Hyperparameters

The settings **you** choose before training, like the step size.

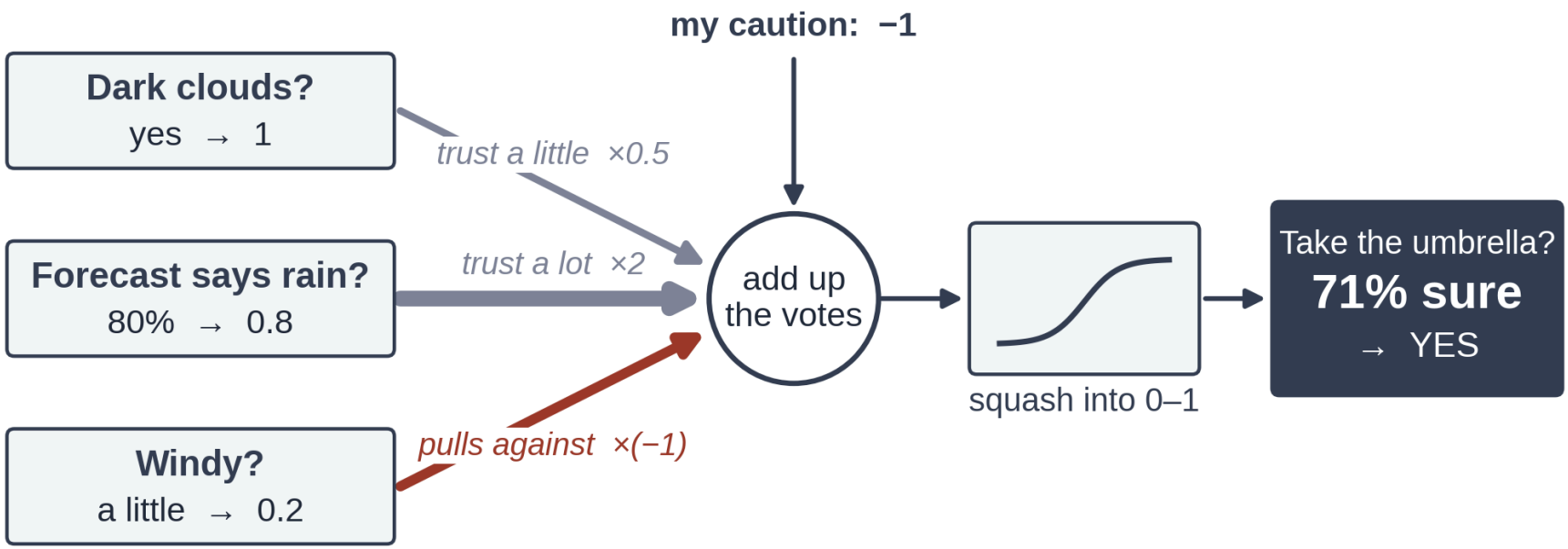
PART 2

Guess, with one neuron

Weigh the clues, add them up, decide



A neuron weighs the clues and decides

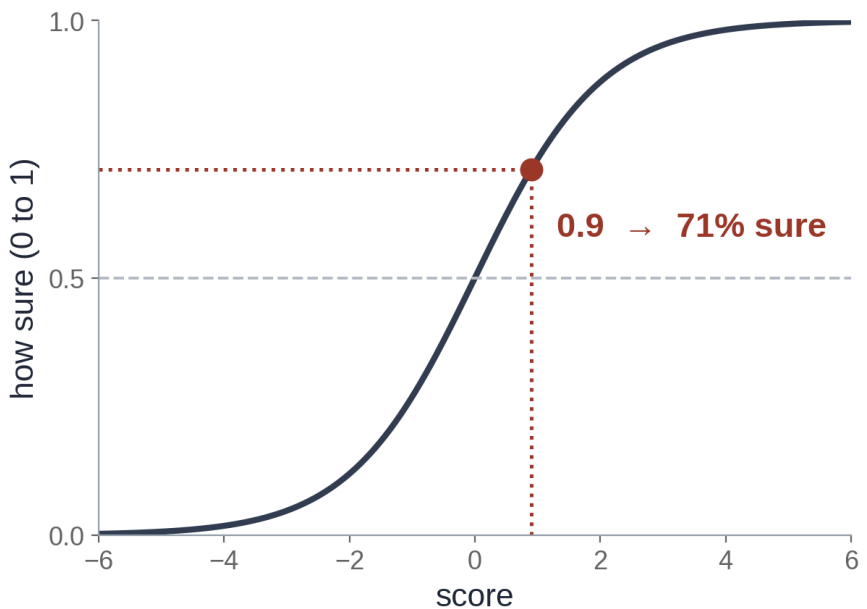


Let’s work one out by hand

clue		trust		vote
1.0	×	0.5	=	0.5
0.8	×	2.0	=	1.6
0.2	×	−1.0	=	−0.2

add the votes	1.9
+ caution (bias)	−1.0
score	**0.9**

squash



One neuron, as a formula

THE FORMULA

$$z = w_1 x_1 + w_2 x_2 + w_3 x_3 + b \quad \hat{y} = \sigma(z)$$

IN WORDS

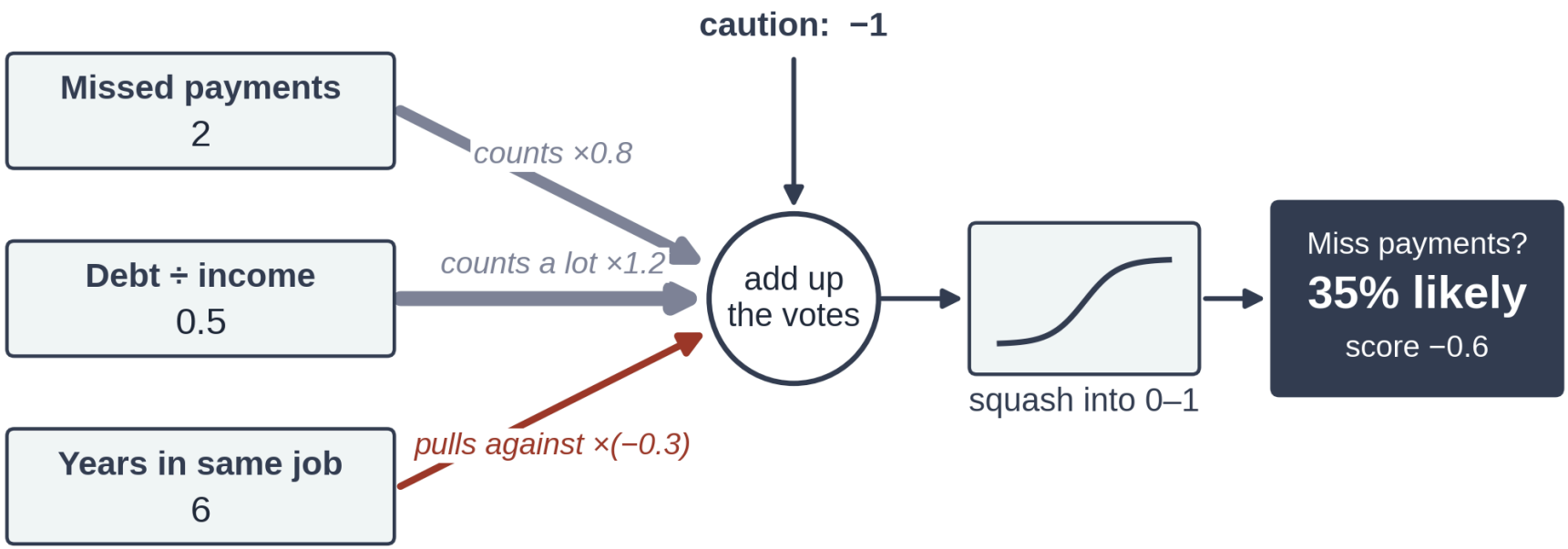
score = trust × clouds + trust × forecast + trust × wind + caution
how sure = squash(score)

WITH NUMBERS

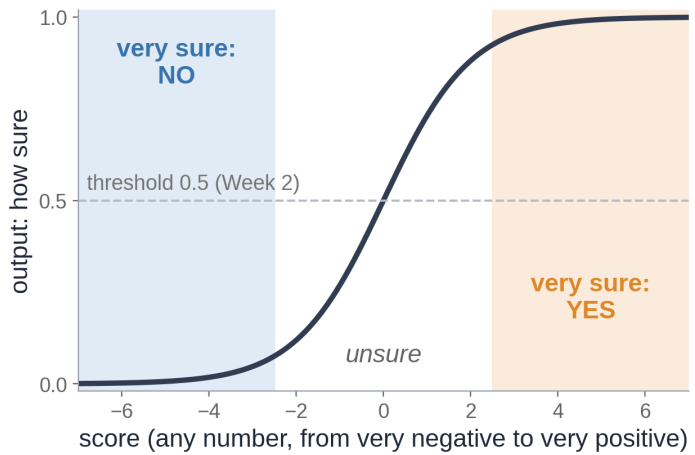
0.9 = 0.5 × 1 + 2 × 0.8 + (−1) × 0.2 + (−1) **0.71 = squash(0.9)**

x = the clues, w = the weights (trust), b = the bias (caution), σ = the squash.

The same neuron, deciding about a loan



The S-curve turns a score into “how sure”



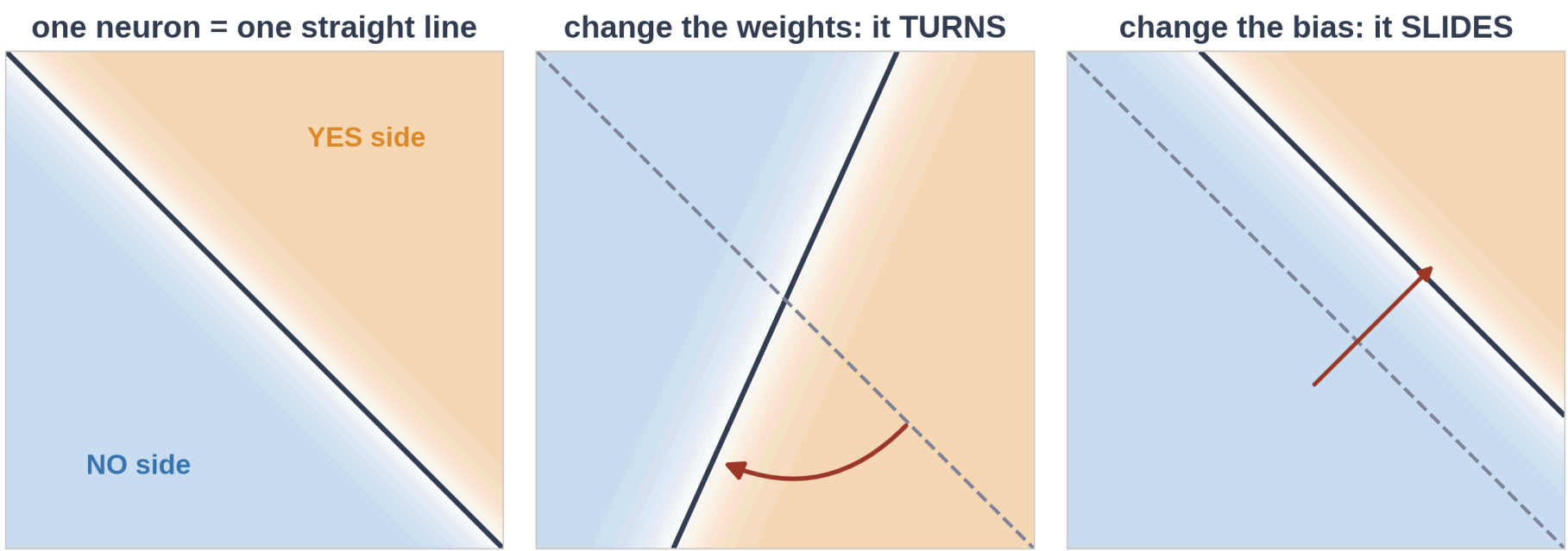
THE FORMULA

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

IN WORDS **how sure = squash(score)**

WITH NUMBERS **0.71 = squash(0.9)**

One neuron draws one straight line

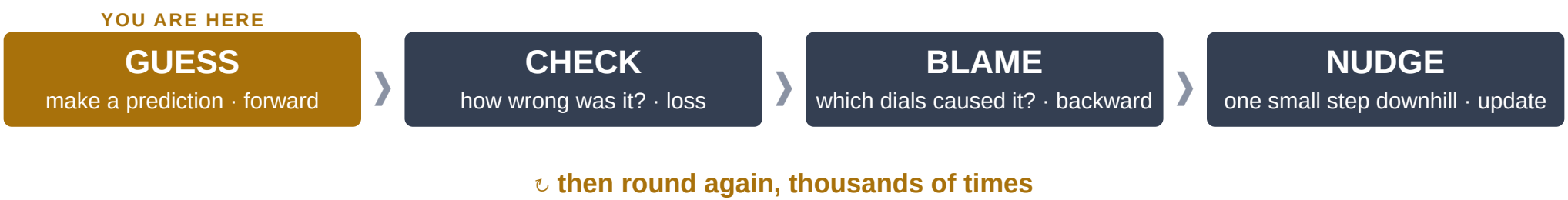


The line is where $z = 0$. The weights turn it; the bias slides it.

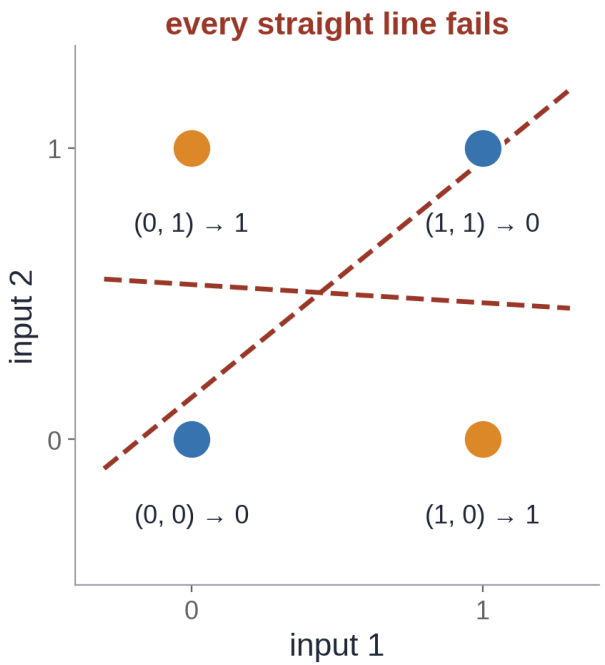
PART 3

Guess, with a network

How it is wired, and why

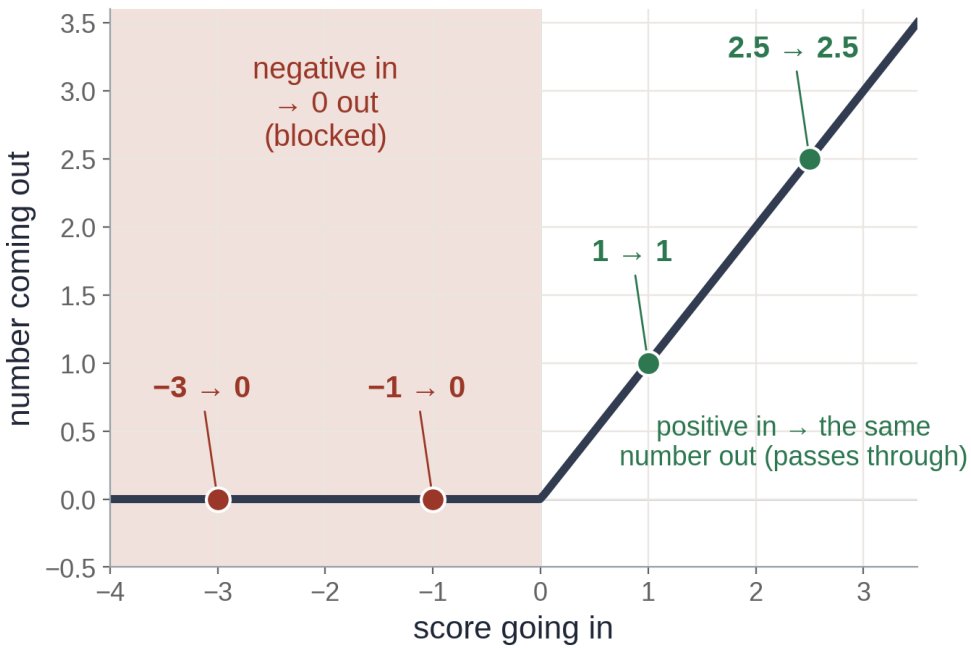


Four dots one neuron cannot sort



No straight line puts both orange dots on one side.

ReLU: let positives through, block negatives



THE FORMULA $\mathrm{ReLU}(z) = \max(0, z)$

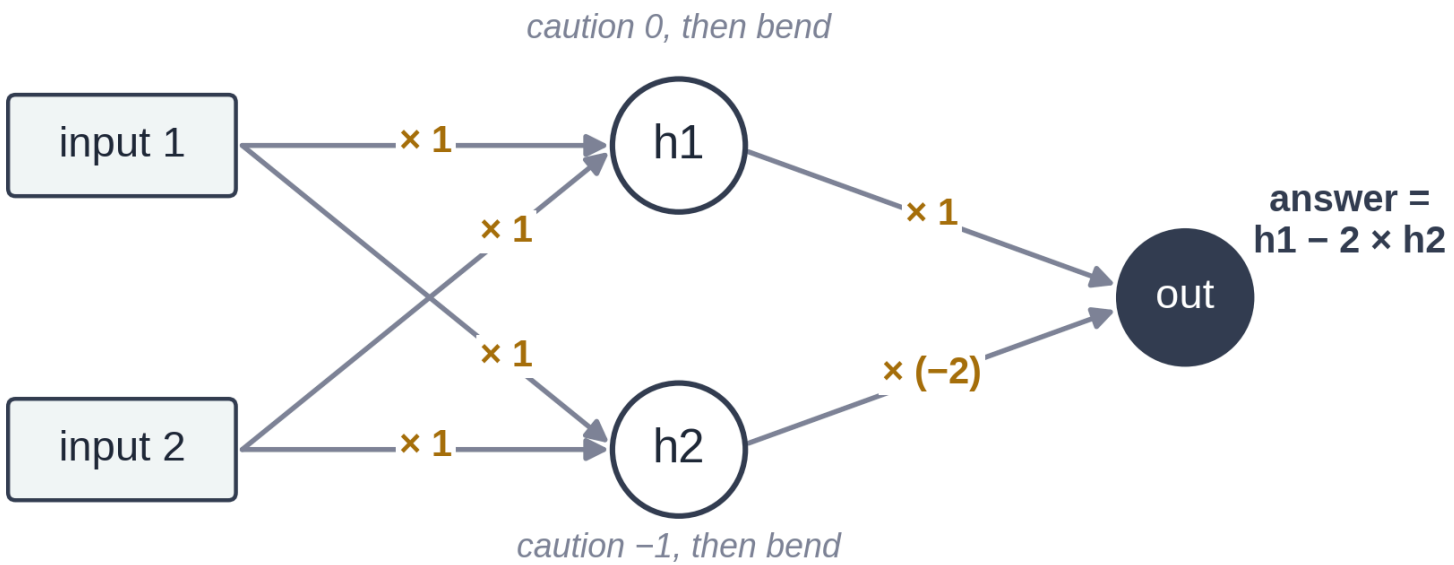
IN WORDS **output = the score if it is positive, otherwise 0**

WITH NUMBERS

ReLU(−3) = 0
ReLU(0.5) = 0.5
ReLU(2) = 2

Like a one-way valve: positive flows through unchanged; negative is stopped at zero.

Add two hidden neurons in the middle



bend = ReLU: a negative number becomes 0, anything else passes through

A hidden layer, as a formula

THE FORMULA

$$h_1 = \mathrm{ReLU}(x_1 + x_2) \quad h_2 = \mathrm{ReLU}(x_1 + x_2 - 1) \quad \hat{y} = h_1 - 2h_2$$

IN WORDS

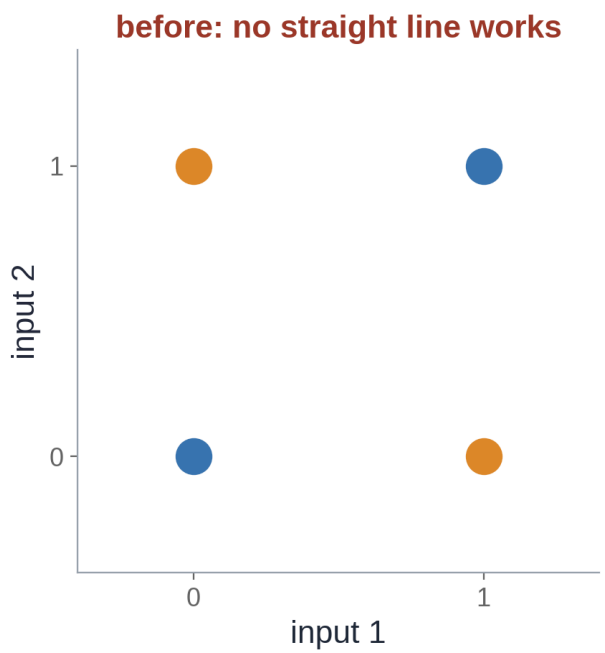
hidden 1 = bend(switch 1 + switch 2) hidden 2 = bend(switch 1 + switch 2 − 1)
answer = hidden 1 − 2 × hidden 2

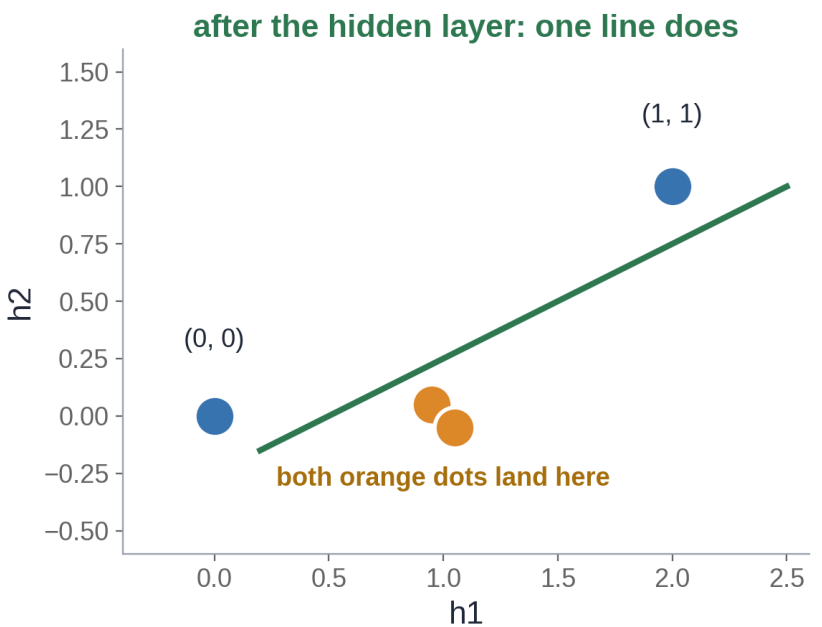
WITH NUMBERS

dot (1, 1): 2 = ReLU(1 + 1) 1 = ReLU(1 + 1 − 1) 0 = 2 − 2 × 1

Any layer is $\boldsymbol{h} = \mathrm{ReLU}(W\boldsymbol{x} + \boldsymbol{b})$: every neuron does its own weighted sum of the same inputs, then bends.

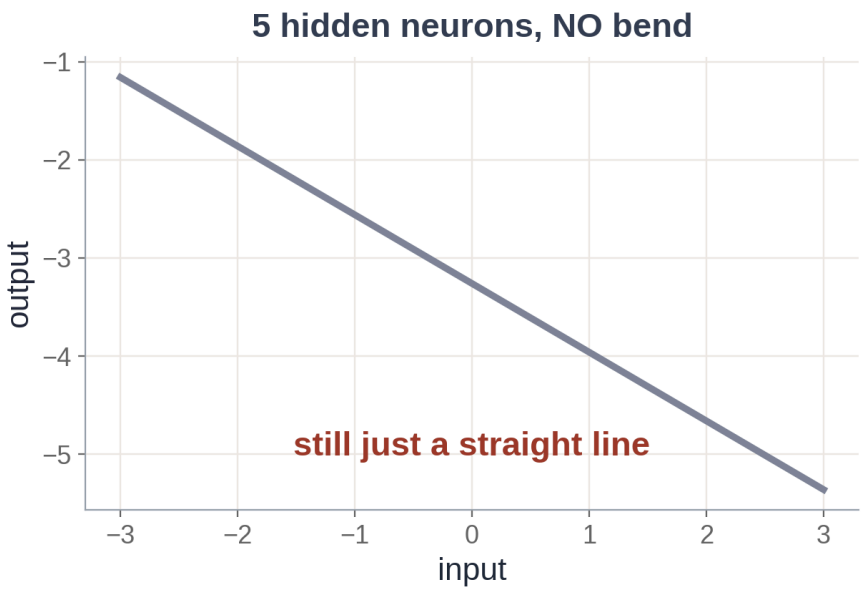
The hidden layer moves the dots

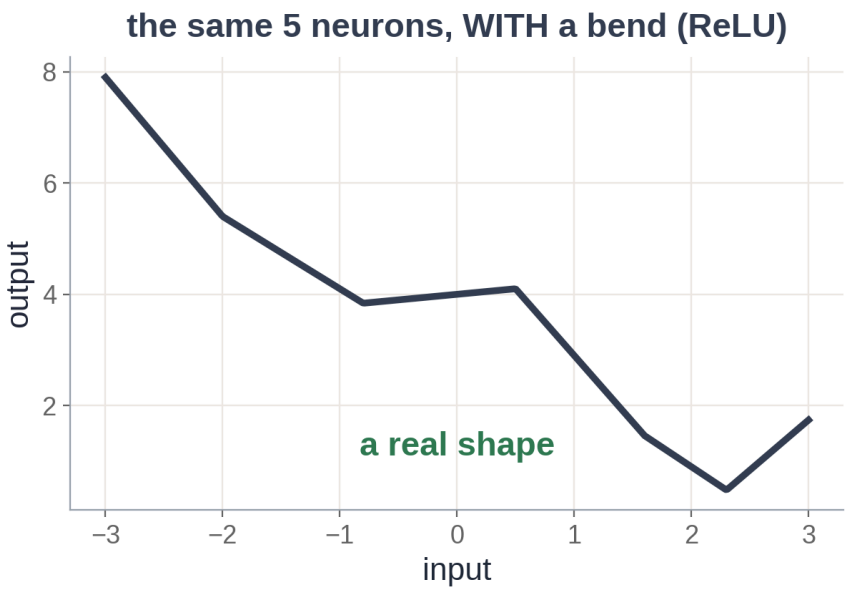




In the new numbers (h_1, h_2), one straight line works.

Why every neuron needs a bend

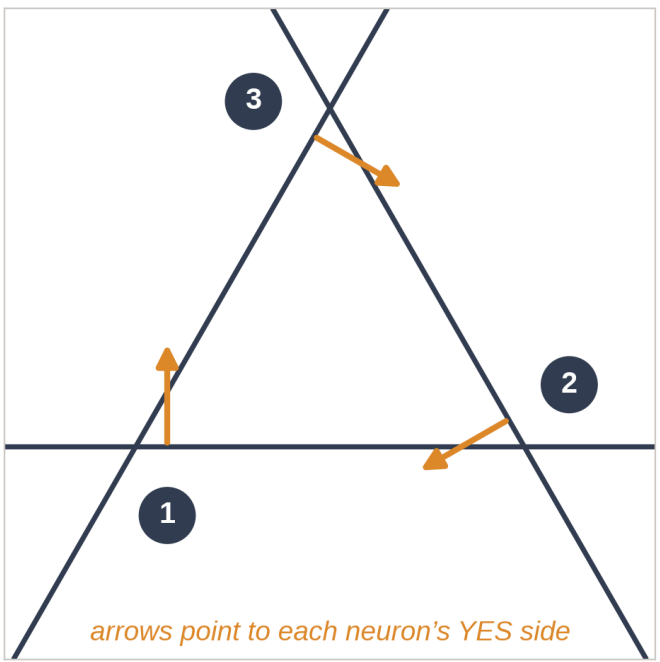




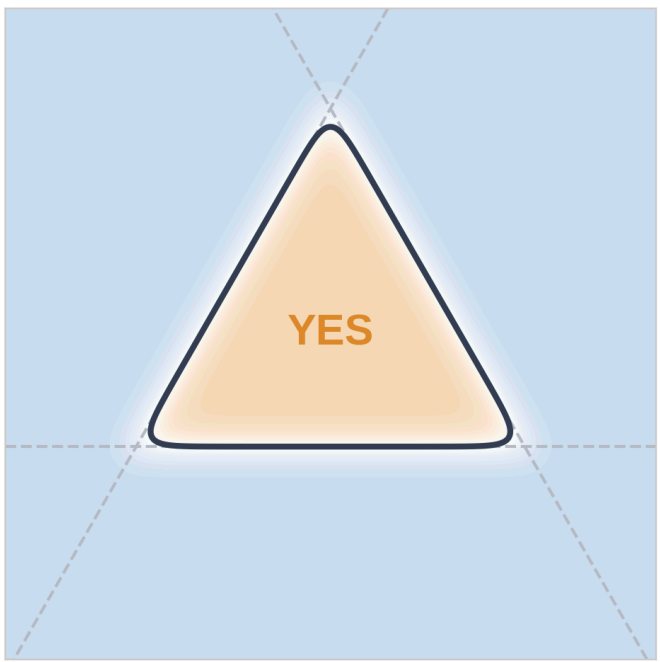
Without ReLU, two layers collapse into one: $W_2(W_1 x) = (W_2 W_1)x$, still a straight line. The bend is the hinge.

More neurons, more lines

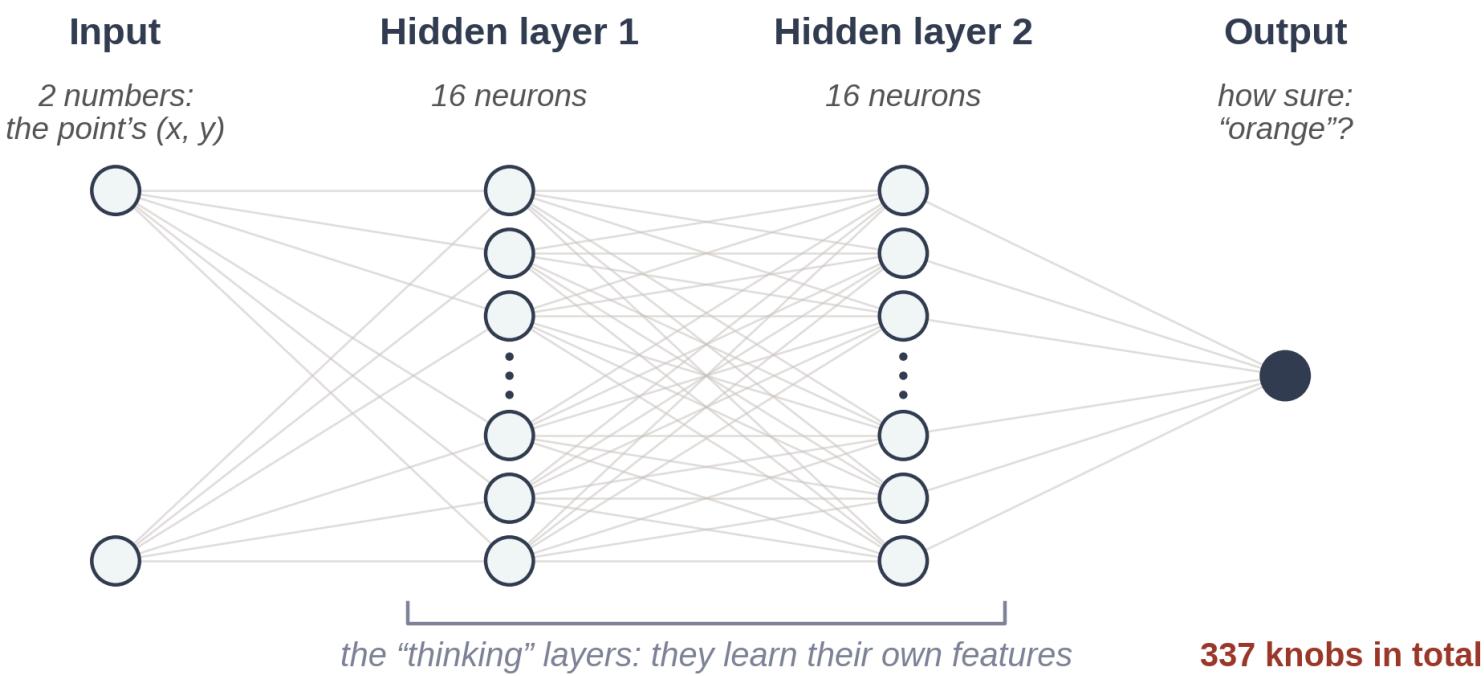
Layer 1: three neurons, three lines



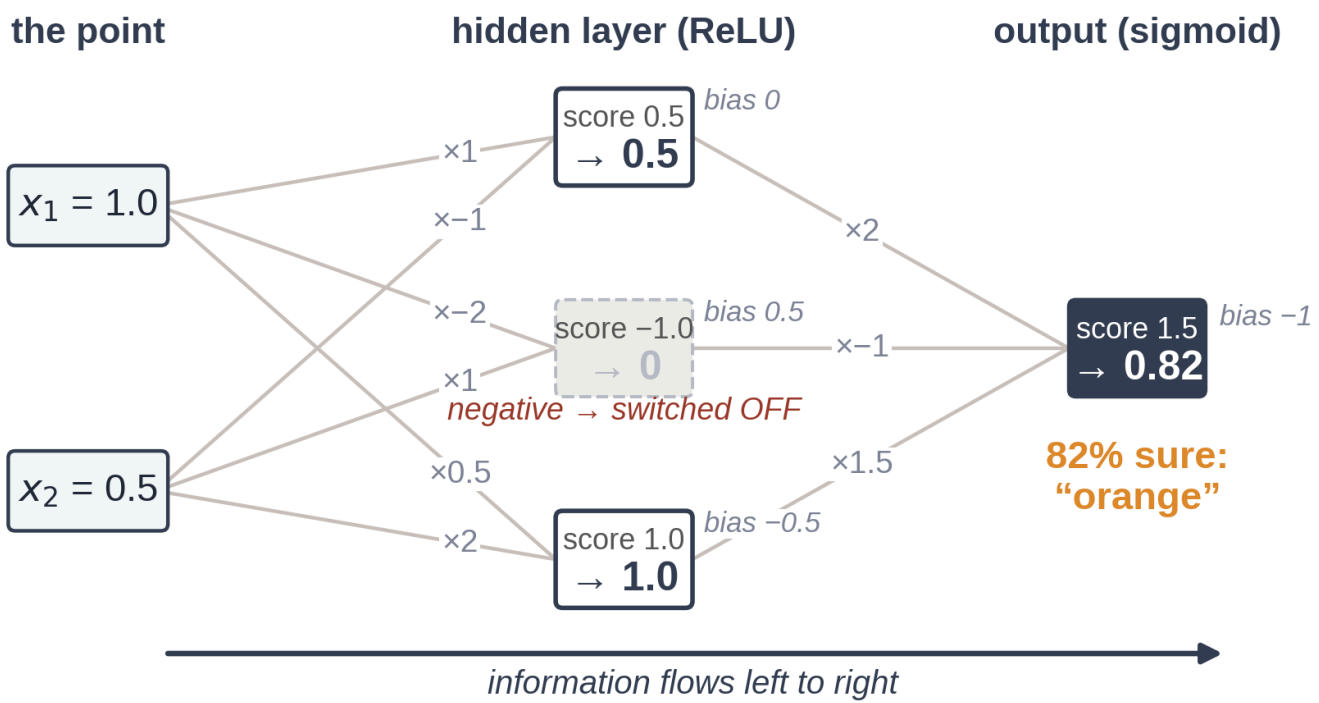
Layer 2: “yes only where all three say yes”



Today’s network



Asking it a question: numbers flow left to right



The whole network, as one formula

THE FORMULA

$$\hat{y} = \sigma(W_3\,\mathrm{ReLU}(W_2\,\mathrm{ReLU}(W_1\boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2) + b_3)$$

IN WORDS

how sure = squash(sum of bends(sum of bends(sum of the dot)))

WITH NUMBERS

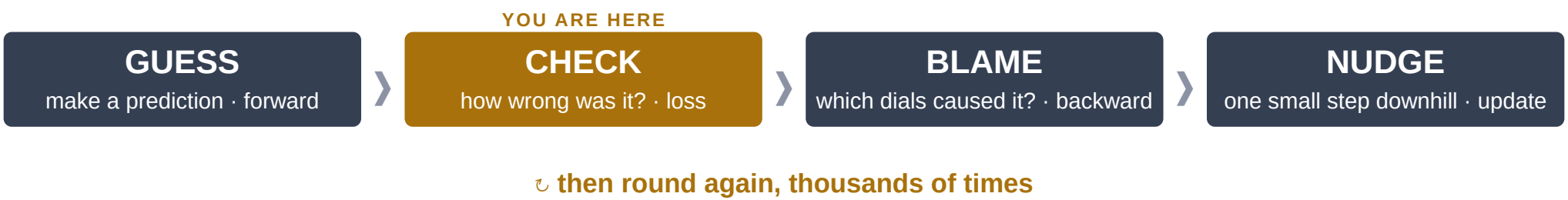
dot → 16 numbers → 16 numbers → how sure it is orange

Read it from the inside out: that is the forward pass, left to right. All the W's and $\boldsymbol{b}$'s together are the 337 dials.

PART 4

Check: how wrong was it?

One number that scores every guess



The loss is like a darts score

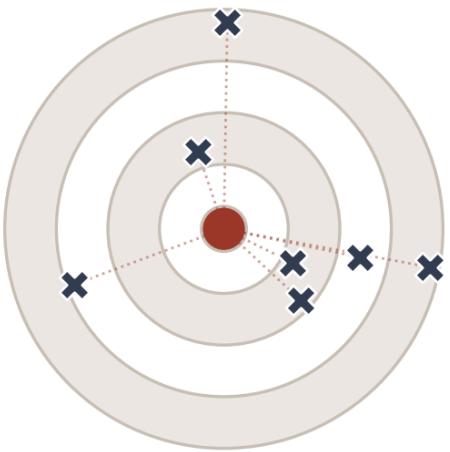
good guesses



small loss

loss = average distance from the bullseye

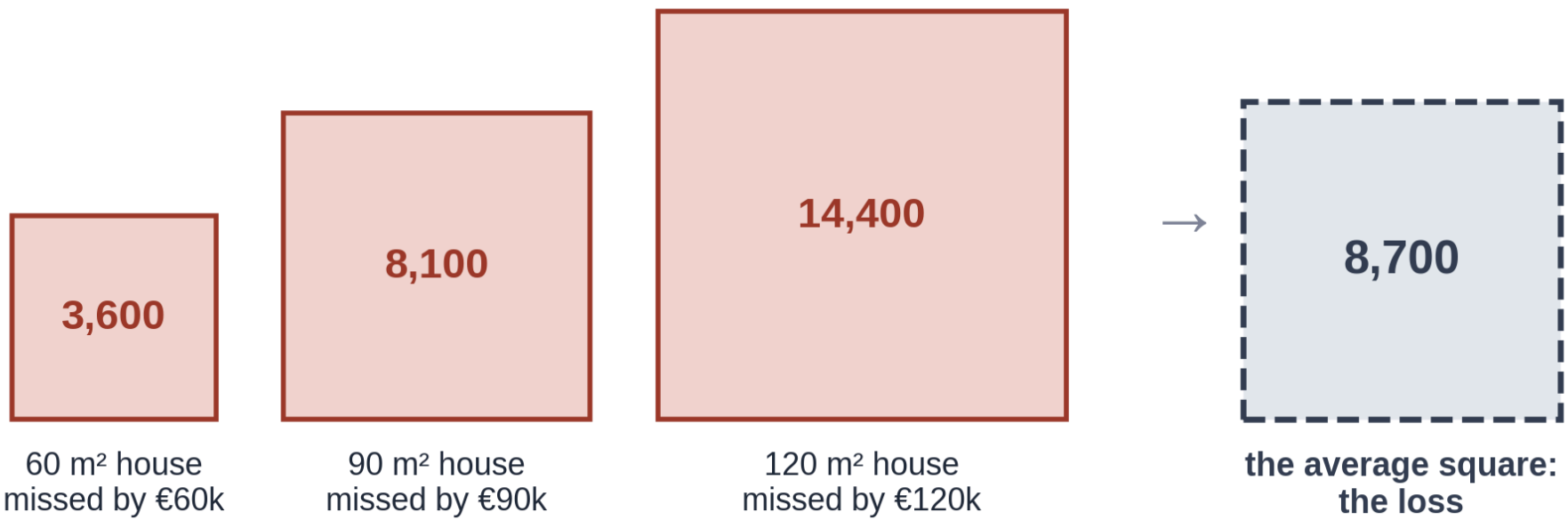
bad guesses



big loss

Lower is better.

Square the misses, then average



The average squared miss, as a formula

THE FORMULA

$$L = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2$$

IN WORDS

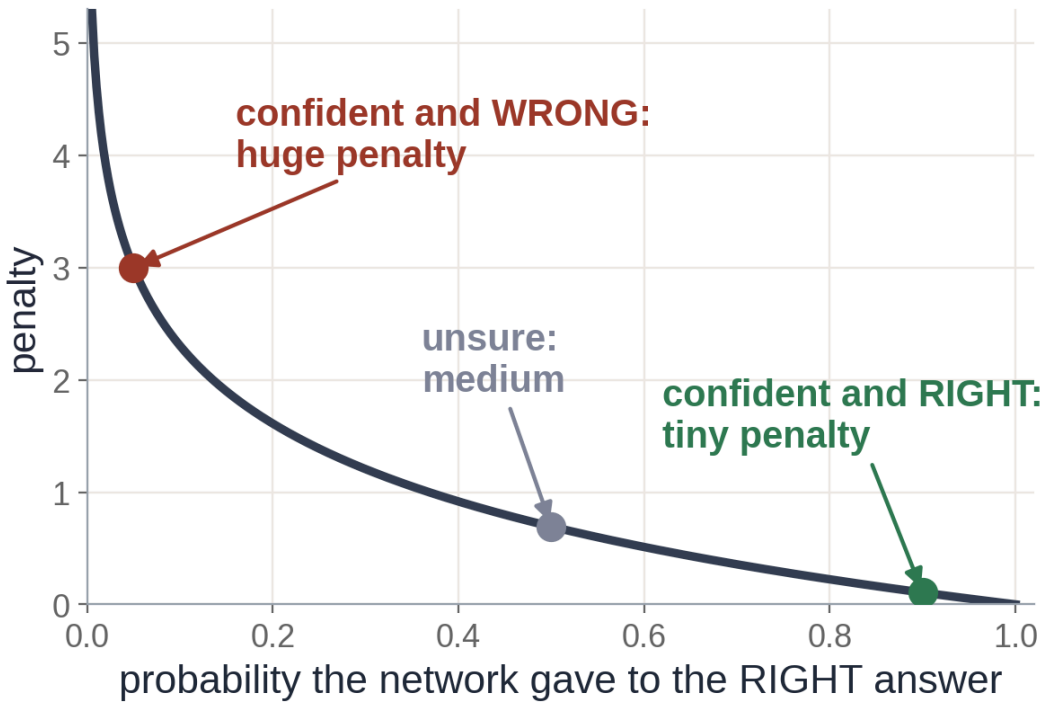
loss = average over all the houses of (guess − real price)²

WITH NUMBERS

8,700 = [(120 − 180)² + (150 − 240)² + (180 − 300)²] ÷ 3

$\sum$ (“sigma”) means add up; n = 3 houses; dial a = 1000.

Confident and wrong costs the most



Cross-entropy, as a formula

THE FORMULA

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

IN WORDS

penalty = − log(how sure it was of the right answer)

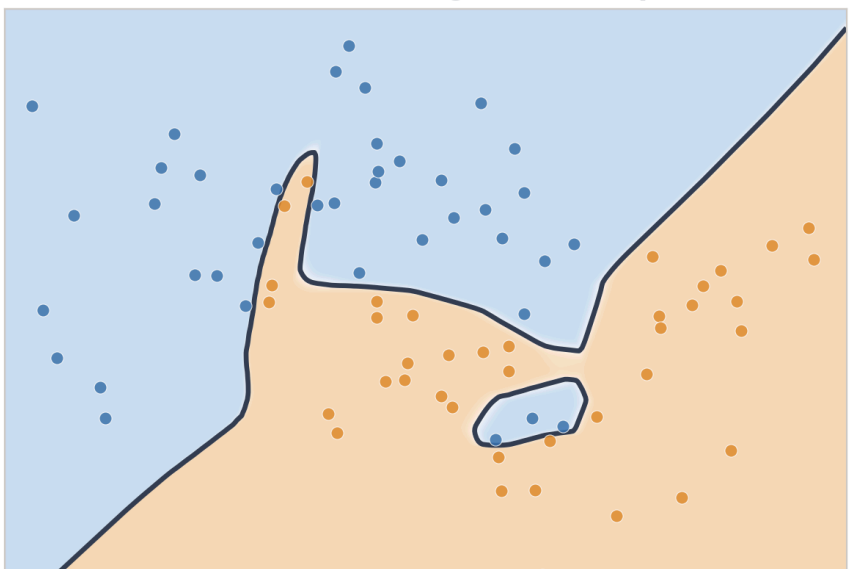
WITH NUMBERS

90% sure and right: 0.1 = − log(0.9) **5% on the right answer: 3.0 = − log(0.05)**

y is 1 for orange and 0 for blue, so it switches on only the half of the formula for the right answer.

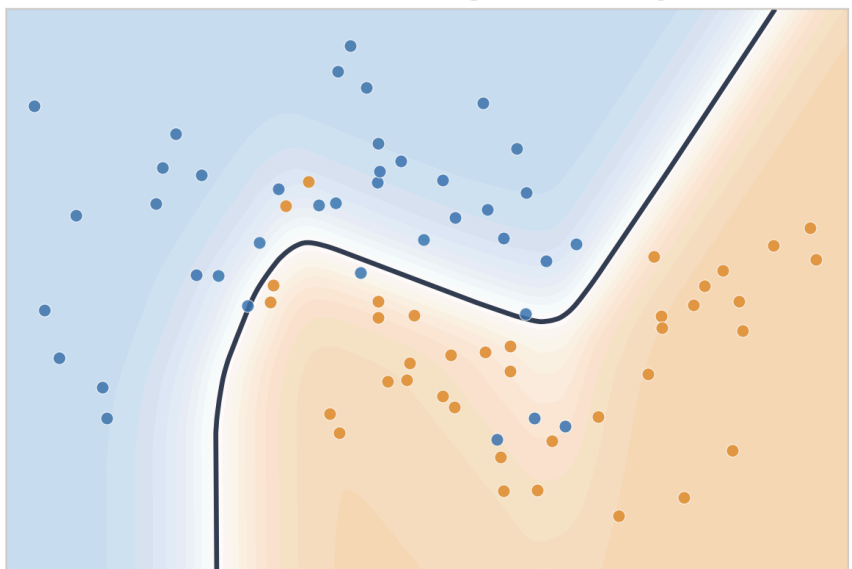
Memorising is not learning

MEMORISED: 85% right on new points



no regularisation: bends round every single dot

UNDERSTOOD: 90% right on new points



weight decay + dropout: a smooth, sensible shape

Always check the loss on dots it has never seen.

PART 5

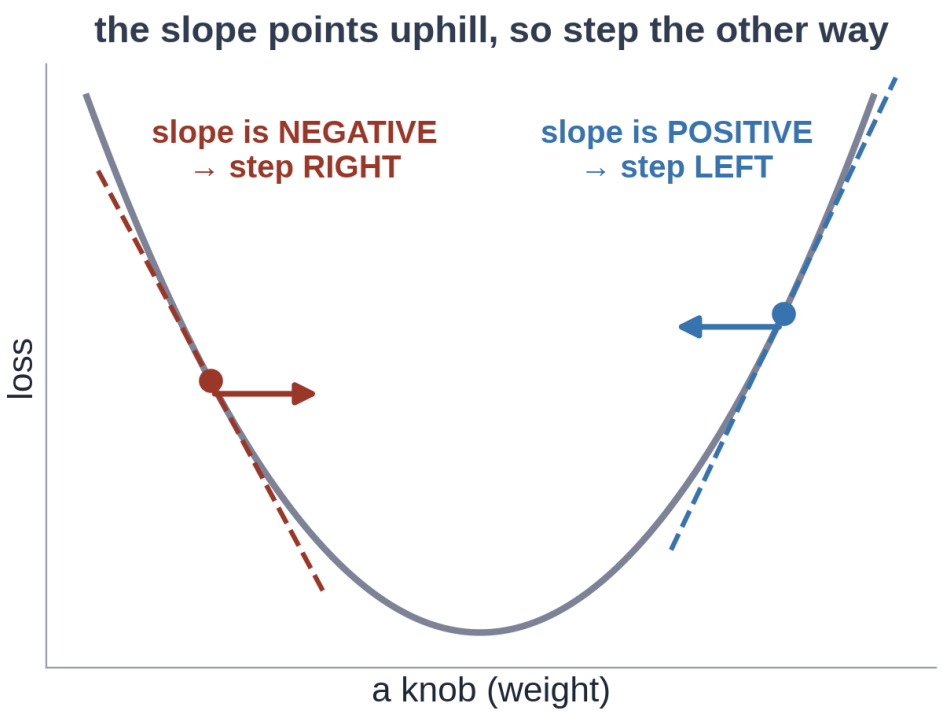
Nudge: walking downhill

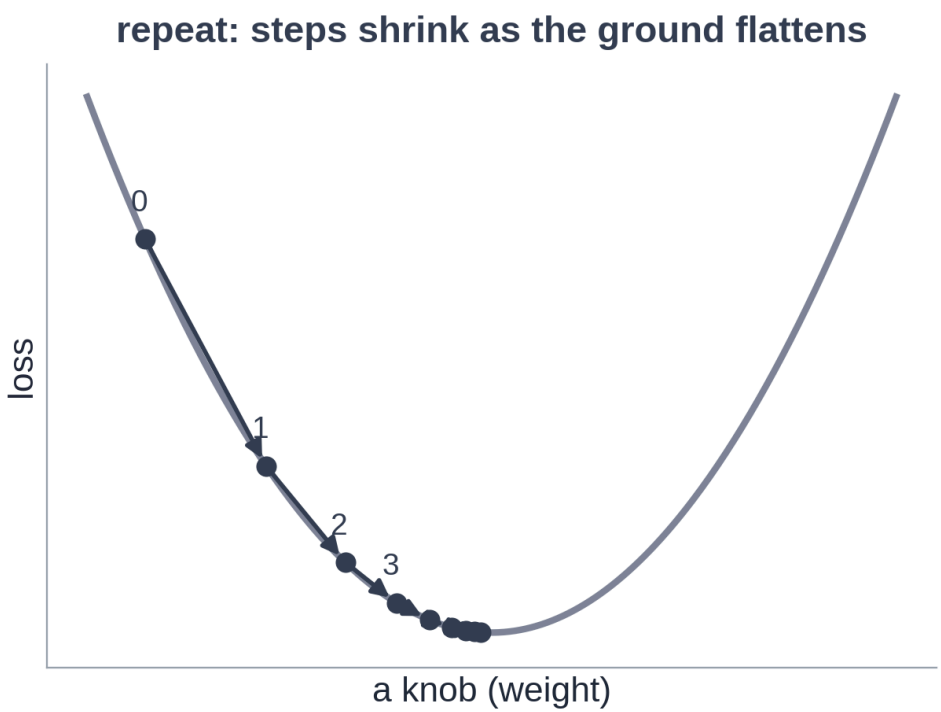
How the dials actually get turned



↻ **then round again, thousands of times**

Feel the slope, step down, repeat





Gradient descent, as a formula

THE FORMULA

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

IN WORDS

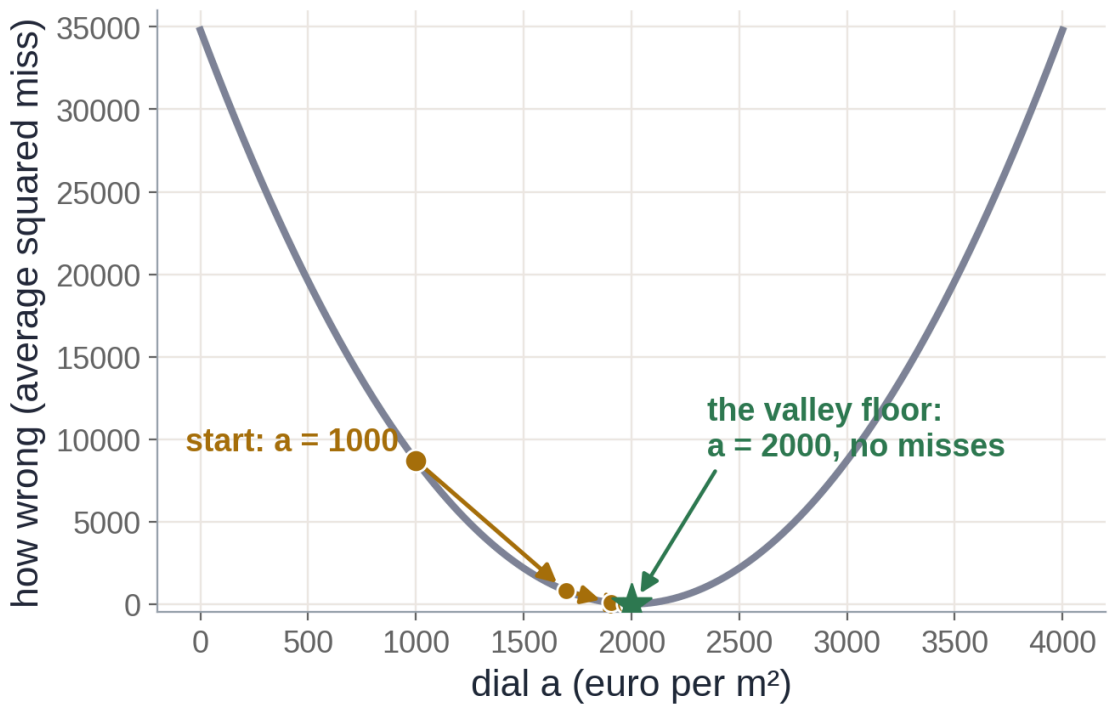
new dial = old dial − step size × slope

WITH NUMBERS

1,696 = 1,000 − 40 × (−17.4)

θ (“theta”) = any dial · η (“eta”) = the step size you choose · $\frac{\partial L}{\partial \theta}$ = the slope. The minus means “go downhill”.

The house dial finds its own way down

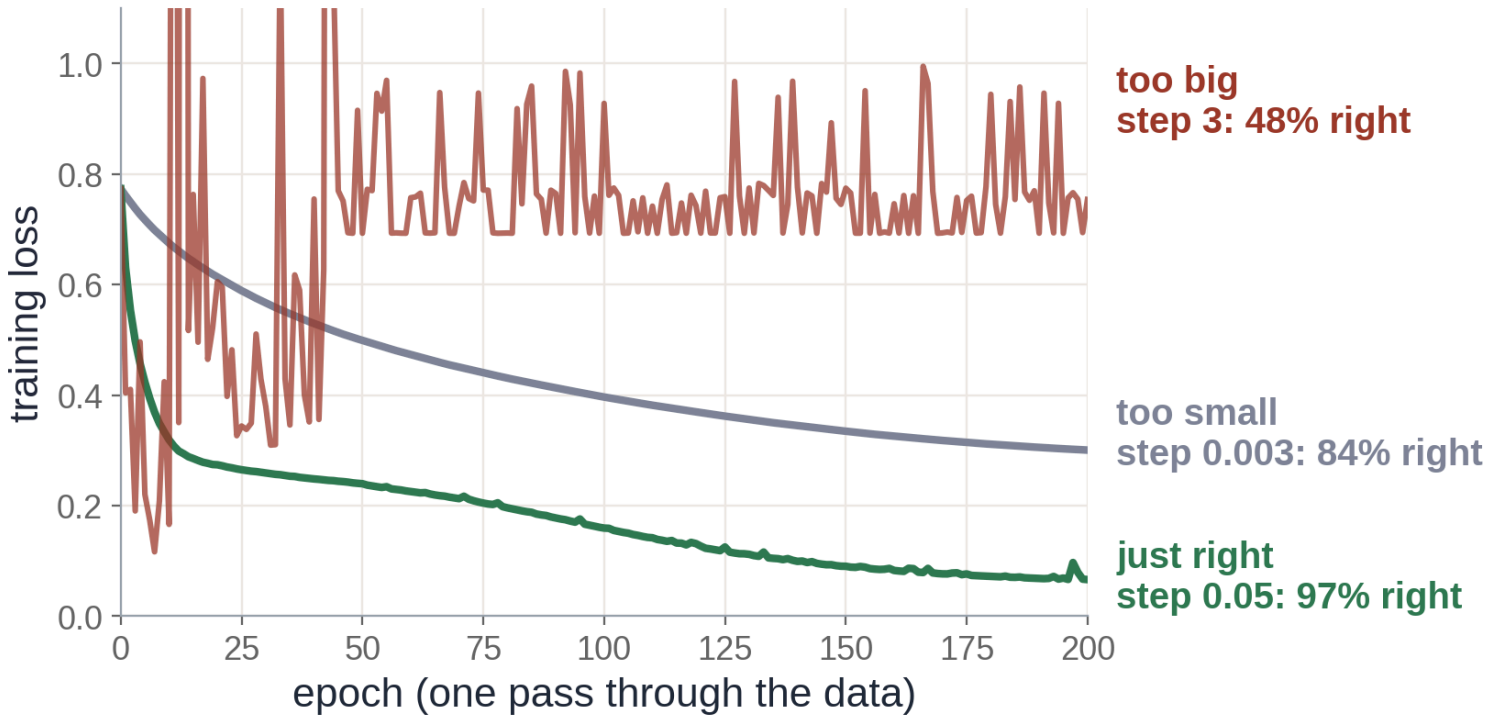


The step size is the whole ballgame



Too small η: crawls. Too big η: overshoots and bounces.

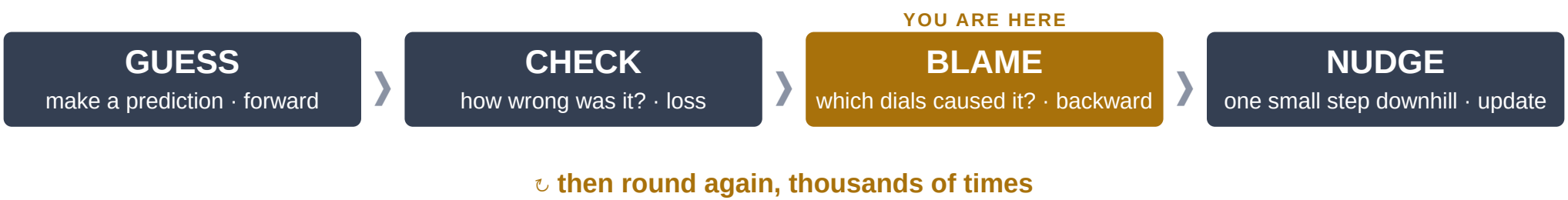
Step size on our real network



PART 6

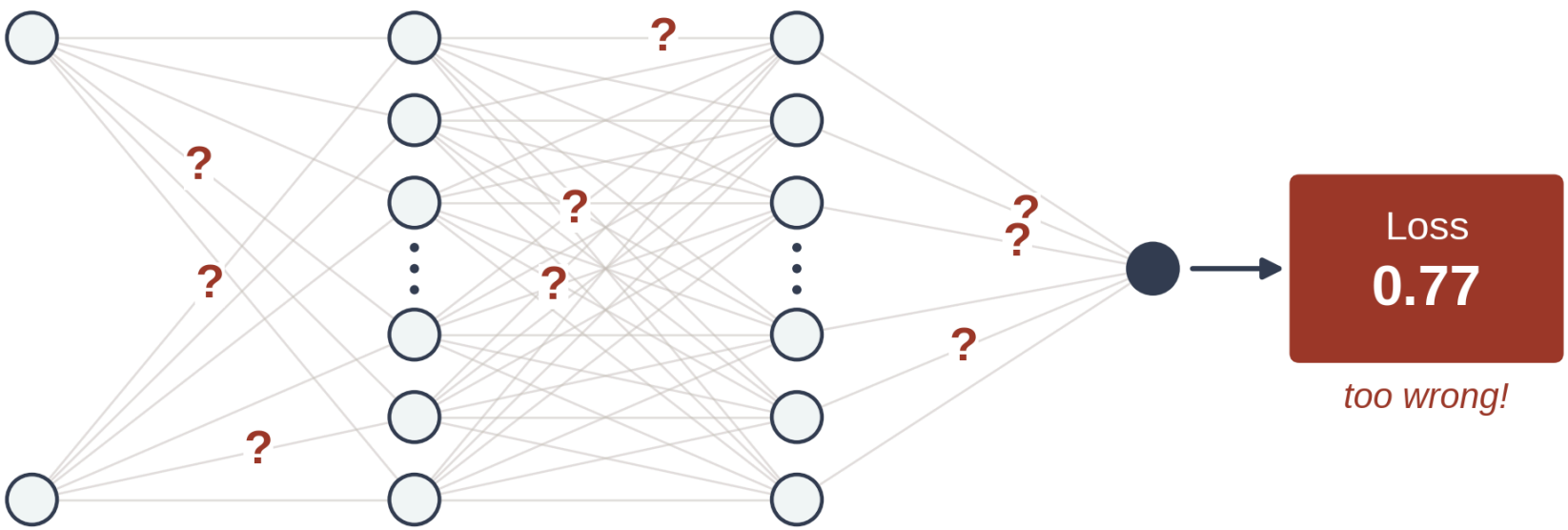
Blame: passing it backwards

Which dial caused the mistake?



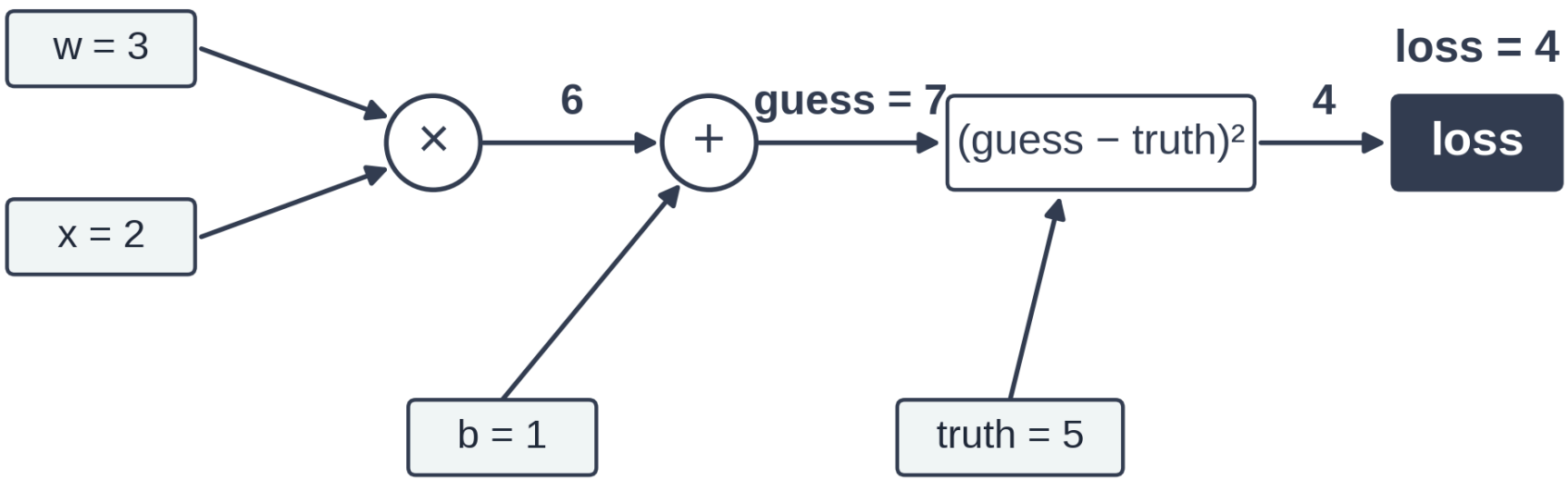
Which dials caused the mistake?

337 knobs. Which ones caused the mistake — which way should each turn, and how far?

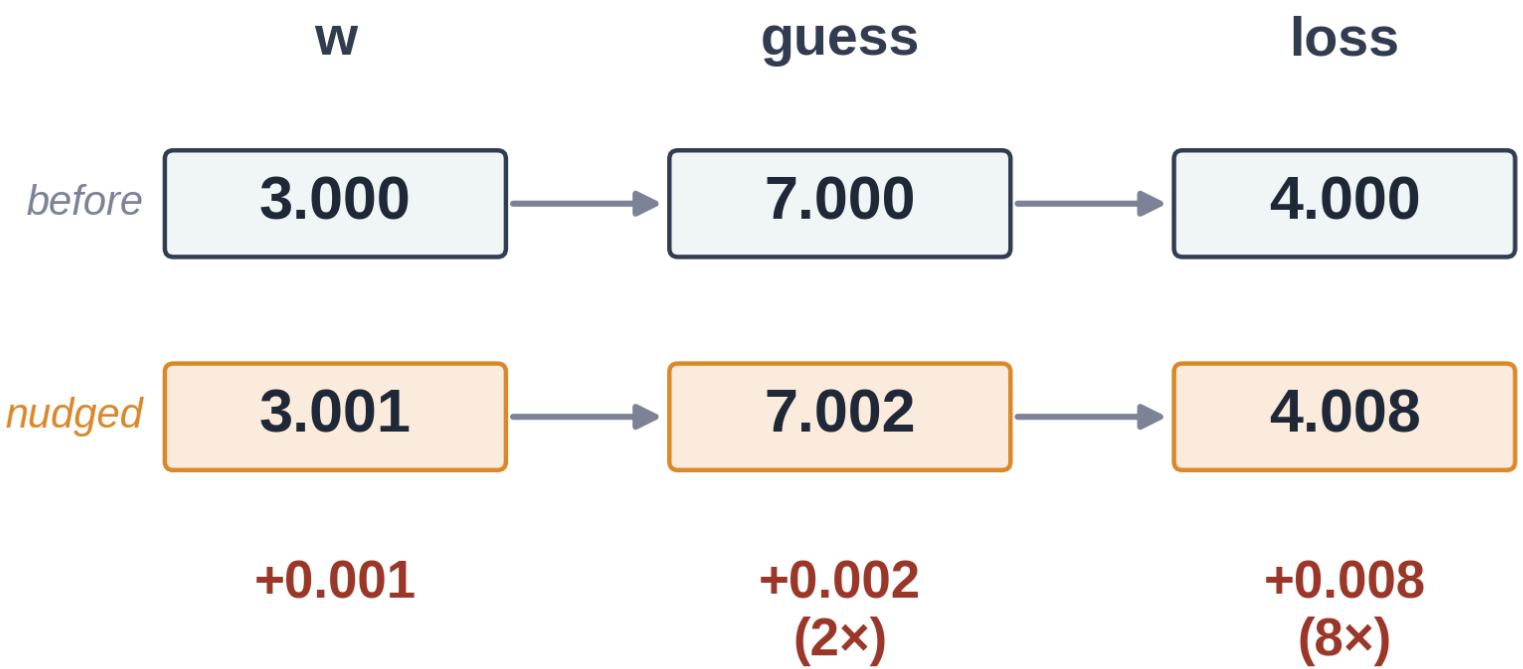


One dial, worked by hand

Forward: put the numbers in and work left to right



Nudge the dial and watch the loss



A tiny nudge

moved the loss
8 times as much

w's blame = 8

Blame, as a formula

THE FORMULA

$$\frac{\partial L}{\partial w} \approx \frac{L(w+\varepsilon) - L(w)}{\varepsilon}$$

IN WORDS

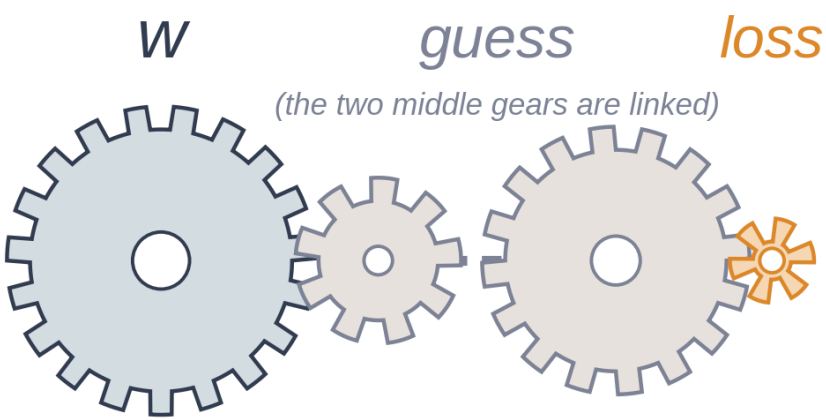
blame of w = (loss after a tiny nudge − loss before) ÷ size of the nudge

WITH NUMBERS

8 = (4.008 − 4.000) ÷ 0.001

ε (“epsilon”) is the tiny nudge. The blame is the same slope gradient descent needs.

Blame multiplies, like gears



w → guess	× 2
guess → loss	× 4
w → loss	**2 × 4 = 8**

multiply along the chain: the ‘chain rule’

The chain rule, as a formula

THE FORMULA

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$

IN WORDS

(how much w moves the loss) = (how much the guess moves the loss) × (how much w moves the guess)

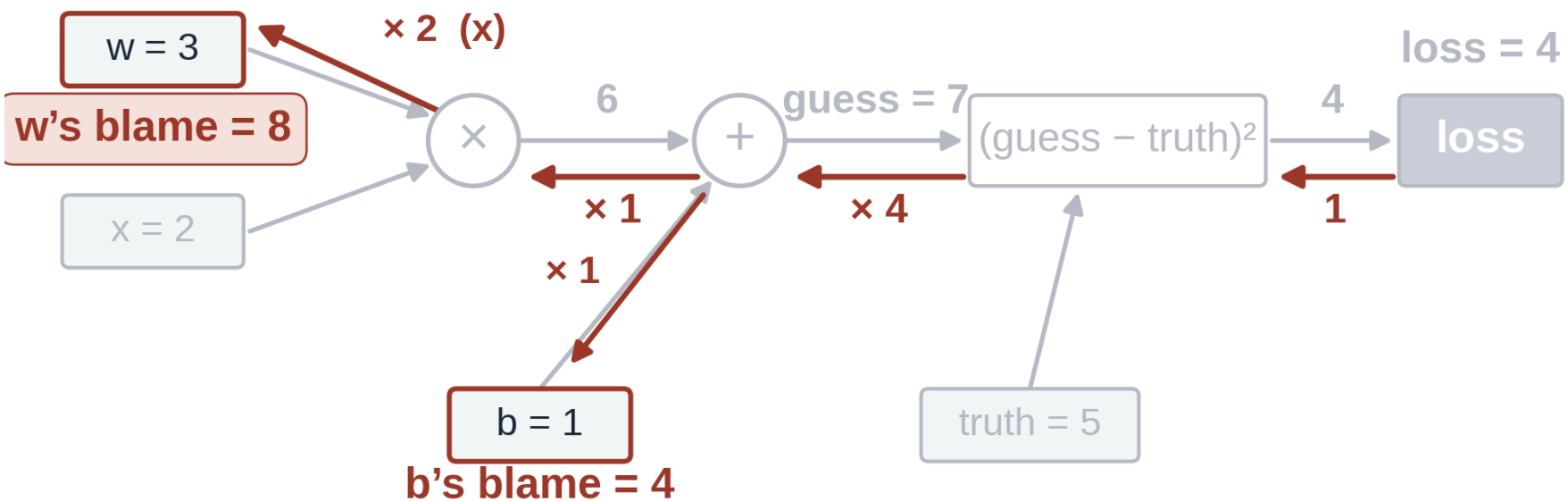
WITH NUMBERS

8 = 4 × 2

Like gears: the turns multiply along the chain. 4 = 2 × the miss of 2; 2 = x.

Backpropagation: walk back from the loss

Backward: start at the loss with 1, and multiply as you walk right to left



In a real network: layer by layer

→ **forward: compute, and REMEMBER each layer's numbers**



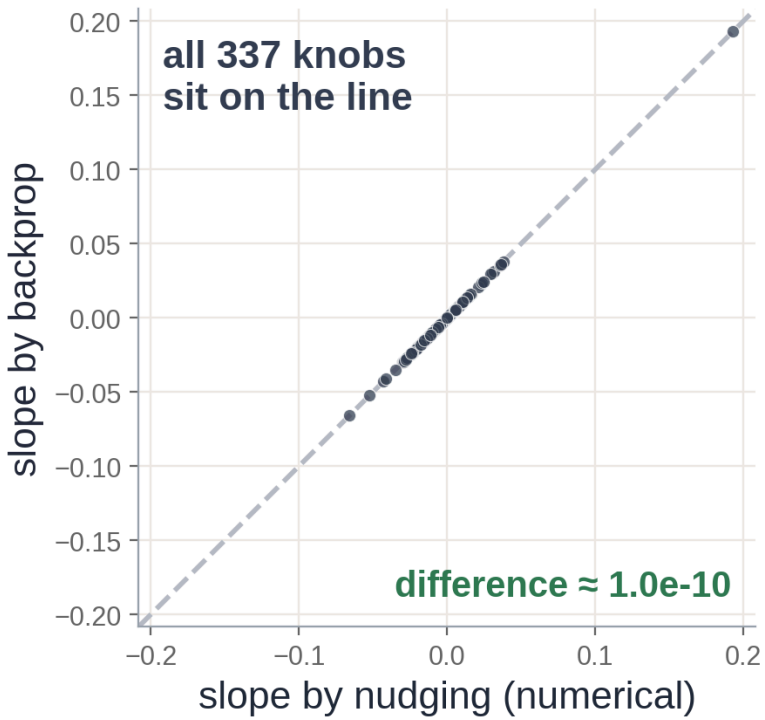
← **backward: pass the blame back, shared out by each connection's weight**

Backpropagation, as three formulas

$\delta_{\text{out}} = \hat{y} - y$	**blame at the end = guess − truth**
$\dfrac{\partial L}{\partial W} = \boldsymbol{h}^T \delta$	**blame of a weight = what came in × blame going out**
$\delta_{\text{back}} = (\delta W^T) \odot \text{ReLU}'(z)$	**blame handed back = blame × the same weights, and nothing through a switched-off neuron**

δ (“delta”) = blame. In the one-dial example: w’s blame = 2 (what came in) × 4 (blame going out) = 8.

The computer checks its own homework



Blame by nudging and blame by backprop agree for all 337 dials.

PART 7

The loop, running

Watch a network learn

GUESS
make a prediction · forward

›

CHECK
how wrong was it? · loss

›

BLAME
which dials caused it? · backward

›

NUDGE
one small step downhill · update

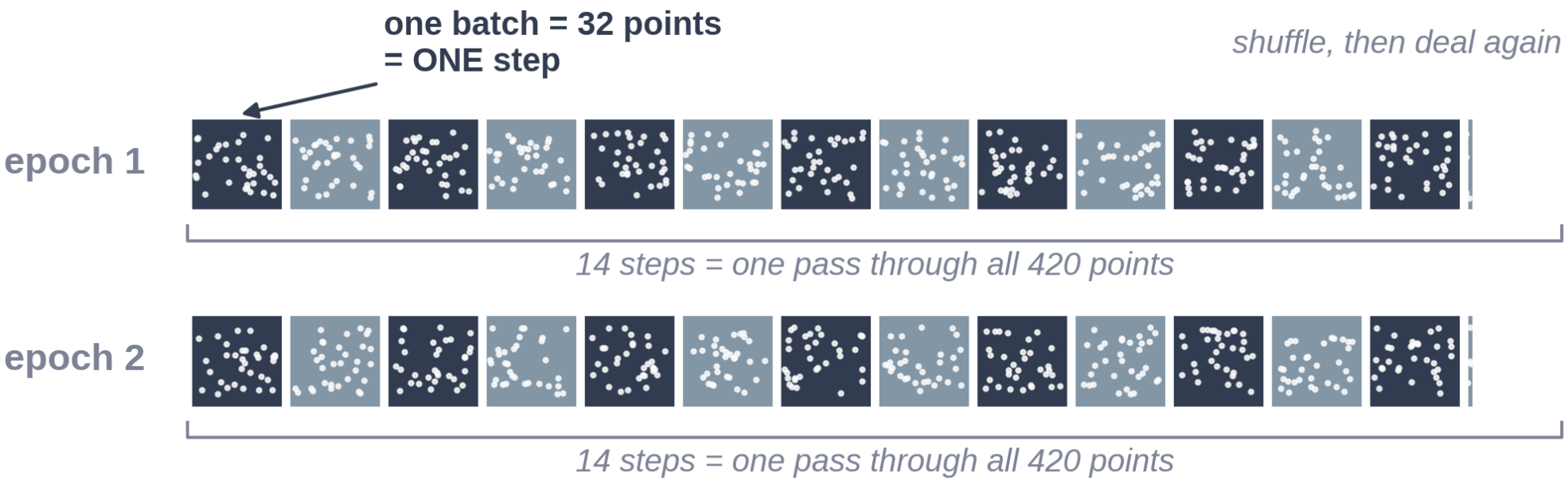
↻ **then round again, thousands of times**

The loop, as four formulas

$\hat{y} = f(x; \theta)$	**1 · GUESS: run the network with today's dials**
$L = \text{loss}(\hat{y}, y)$	**2 · CHECK: one number for how wrong**
$\frac{\partial L}{\partial \theta}$	**3 · BLAME: every dial's blame, walking back**
$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$	**4 · NUDGE: every dial a small step downhill**

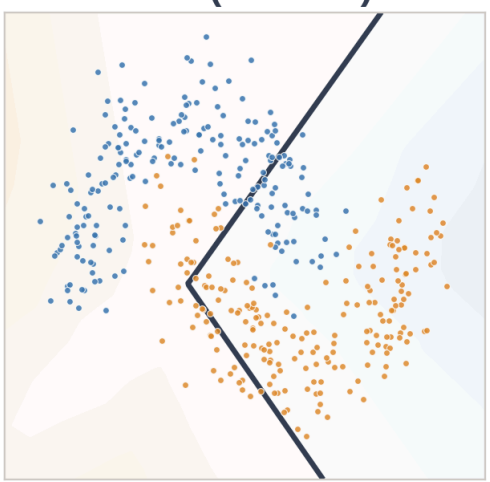
Repeat for every spoonful of 32 dots, 200 times through the data: 2,800 times round the loop.

A spoonful, not the whole pot



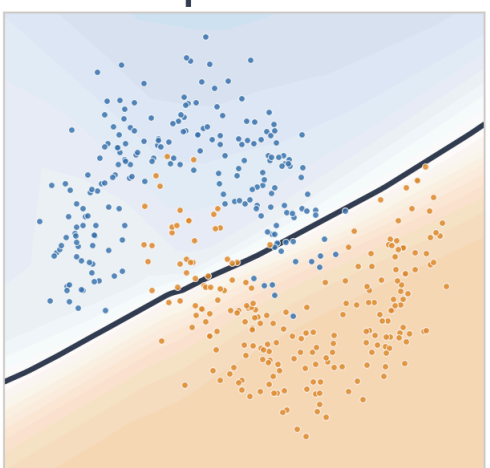
Watch it learn

start (random)



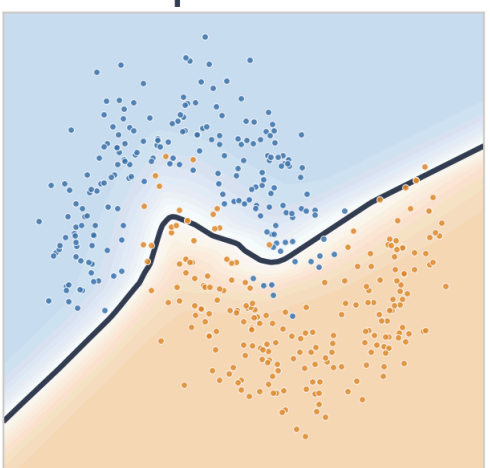
24% right on new points

epoch 10



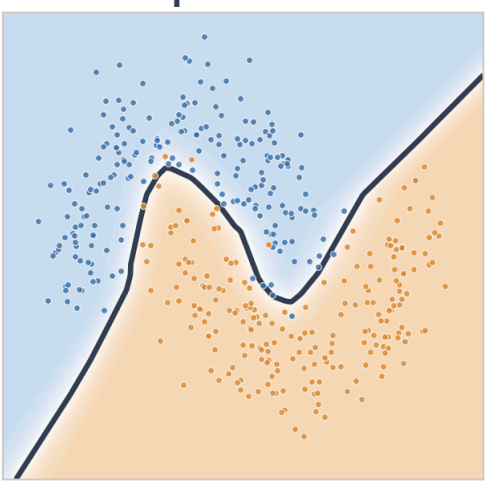
84% right on new points

epoch 100



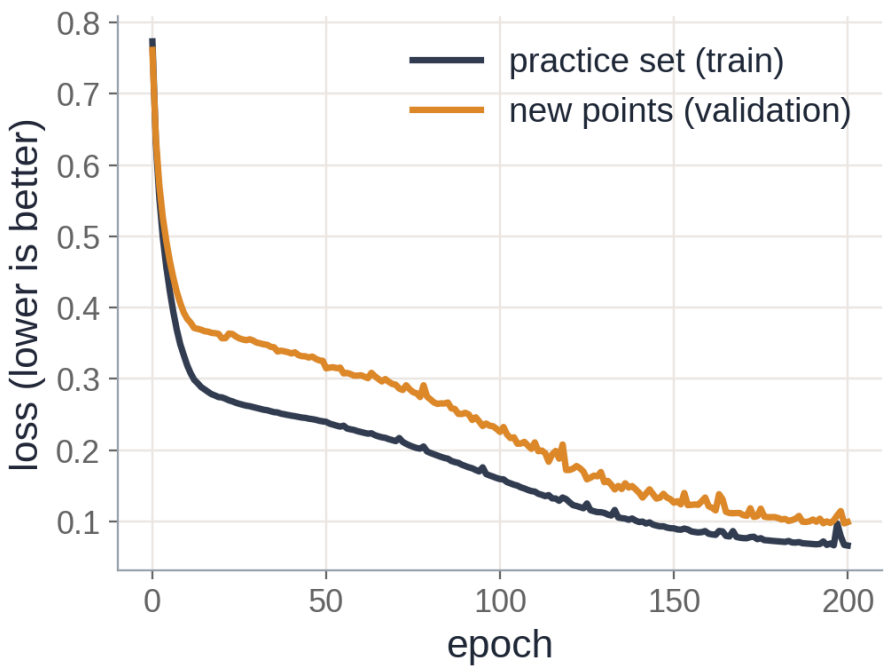
91% right on new points

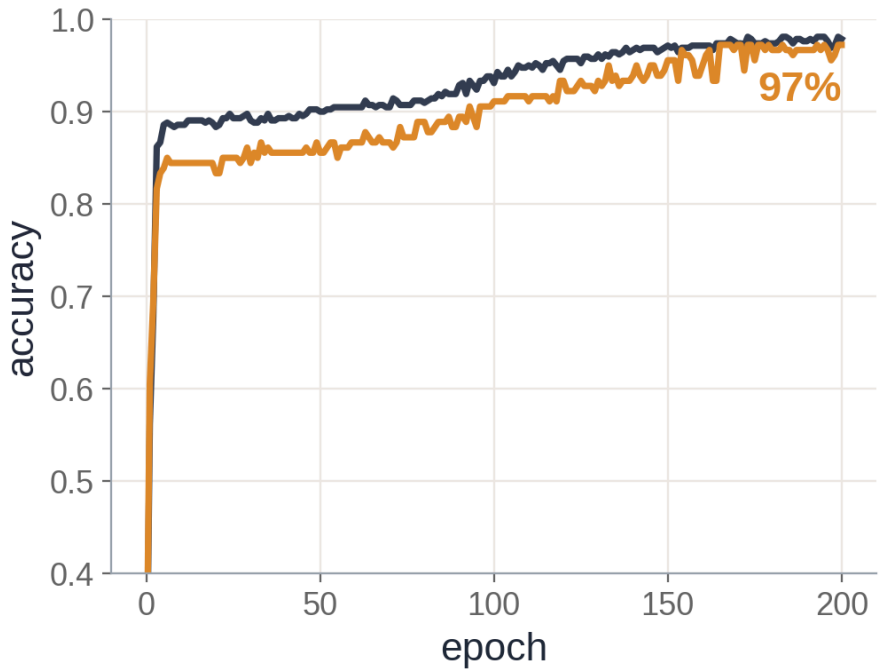
epoch 200



97% right on new points

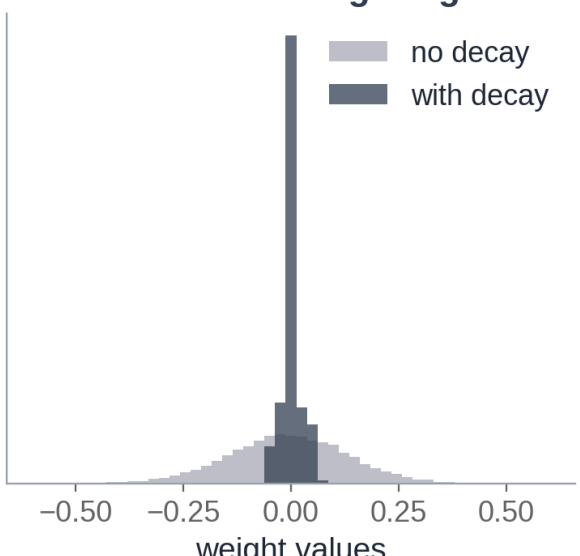
A healthy run



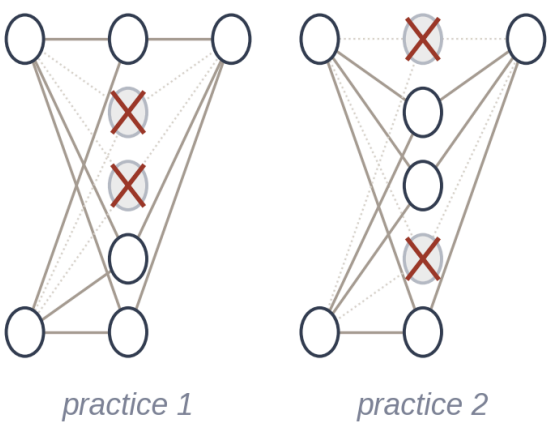


Three cures for memorising

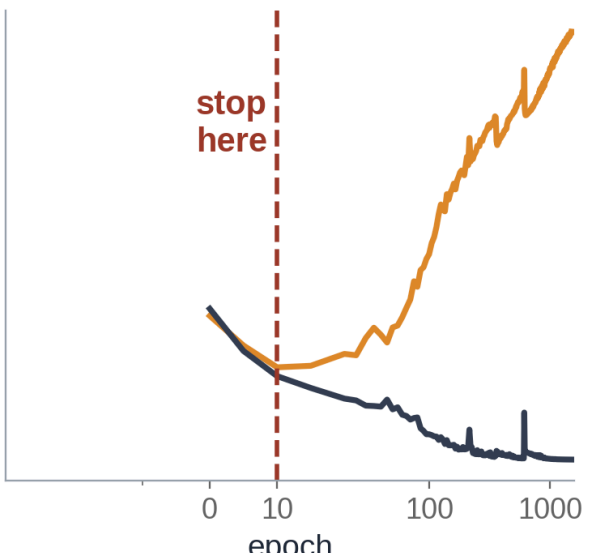
Weight decay
a small tax on big weights



Dropout
random neurons sit out each practice



Early stopping
take the cake out when it's done



Weight decay, as a formula

THE FORMULA

$$L_{\text{total}} = L + \frac{\lambda}{2} \sum w^2$$

IN WORDS

total loss = how wrong + tax × (every weight squared, added up)

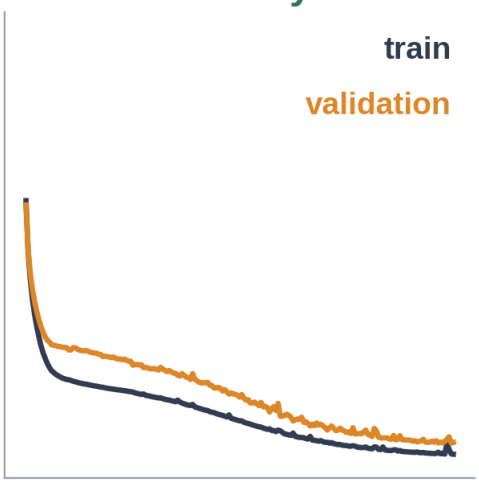
WITH NUMBERS

tax λ = 0.01 in our run: big weights cost more, so the shape stays smooth

λ (“lambda”) = how heavy the tax is; you choose it. This is Week 2’s ridge penalty.

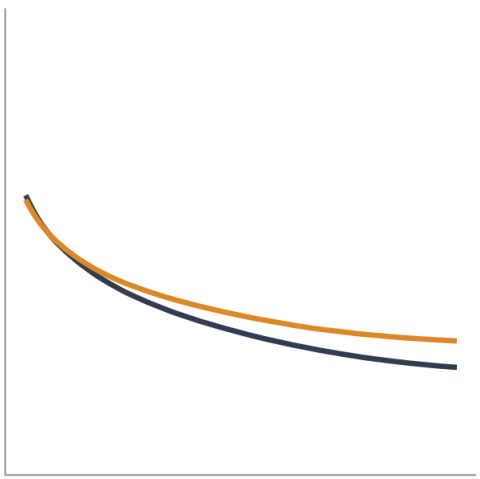
Read the curves like a doctor

Healthy



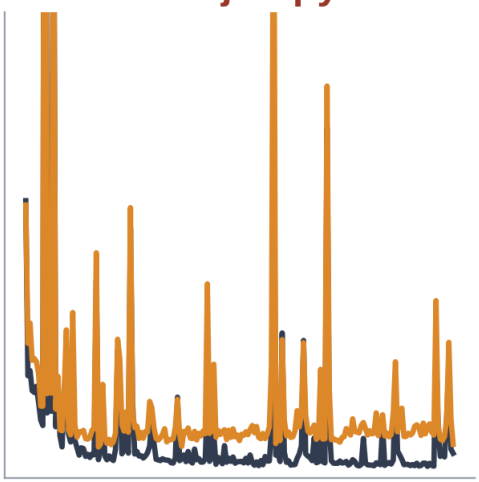
→ keep going

Too slow



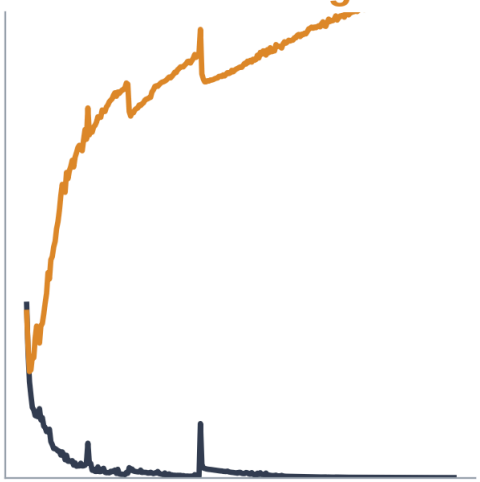
→ raise the learning rate

Too jumpy



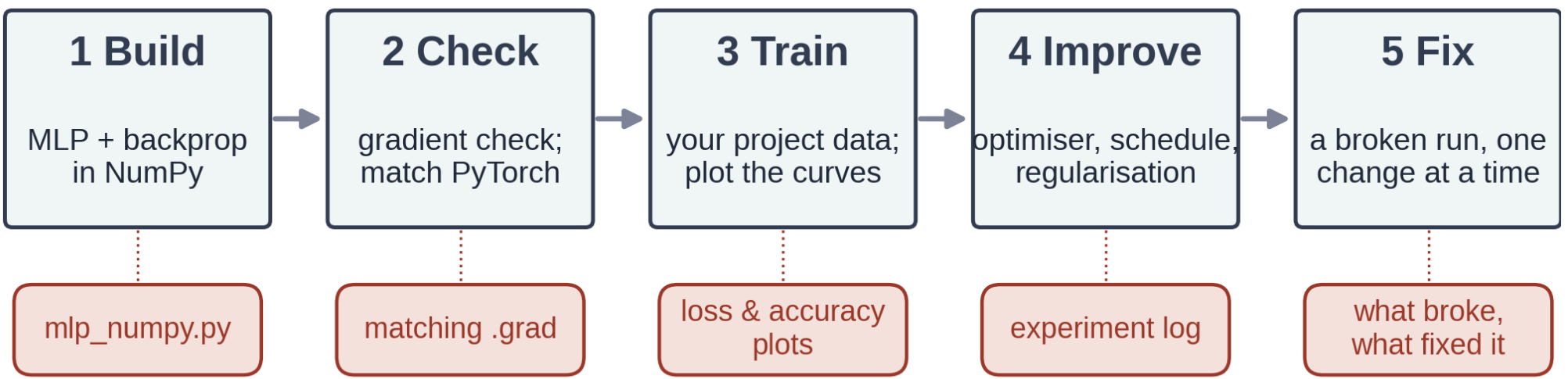
→ lower the learning rate

Memorising



→ regularise / stop early

Today’s lab



hand in



↻ **then round again, thousands of times**

Guess. Check. Blame. Nudge.

Questions?